

03 · TF-IDF

Bag-of-Words (02_bag-of-words.ipynb) infló la similitud entre frases sin relación porque ambas compartían palabras muy comunes como "el". **TF-IDF** corrige eso: un término importa si aparece mucho **en este documento** y poco **en todo lo demás**.

- **TF** (frecuencia en el documento): sube el peso de lo que el texto repite.
- **IDF** (rareza en el corpus): hunde el peso de lo que aparece en todos.
- **peso final = TF × IDF**.

```
corpus = {  
    "D1": "el gato duerme en el sofá",  
    "D2": "el perro duerme en la alfombra",  
    "D3": "el sismo sacudió la costa",  
}
```

IDF a mano sobre el corpus (N = 3)

$$IDF(t) = \log\left(\frac{1 + N}{1 + df(t)}\right) + 1$$

- N : total de documentos (3, así que $1 + N = 4$ en todas las filas).
- $df(t)$: en cuántos de esos documentos aparece el término t .
- El logaritmo comprime la escala; el **+1** evita que un término quede en cero (el mismo suavizado que usa `TfidfVectorizer` por defecto).

Lo único que cambia de una fila a otra es df : cuantos más documentos contienen el término, mayor el denominador y menor el resultado.

```
import numpy as np  
import pandas as pd  
  
tokens_por_doc = {doc_id: set(texto.split()) for doc_id, texto in corpus.items()}  
N = len(corpus)  
  
def idf(termino):  
    df = sum(termino in tokens for tokens in tokens_por_doc.values())  
    return np.log((1 + N) / (1 + df)) + 1, df  
  
filas = []  
for termino in ["el", "duerme", "gato"]:  
    valor, df = idf(termino)  
    filas.append({"término": termino, "df": df, "IDF": round(valor, 3)})  
pd.DataFrame(filas)
```

	término	df	IDF
0	el	3	1.000
1	duerme	2	1.288
2	gato	1	1.693

"el" está en los 3 documentos → IDF mínimo posible (1.000), no distingue. "gato" está en 1 solo → IDF máximo en este corpus (1.693), muy informativa.

Matriz TF-IDF con scikit-learn

`TfidfVectorizer` combina TF × IDF y normaliza cada fila para que todos los vectores midan 1 (por eso el resultado ya no son conteos enteros).

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
X_tfidf = tfidf_vectorizer.fit_transform(corpus.values())

matriz_tfidf = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out(),
    index=corpus.keys(),
).round(2)
matriz_tfidf
```

	alfombra	costa	duerme	el	en	gato	la	perro	sacudió	sismo	sofá
D1	0.00	0.0	0.36	0.55	0.36	0.47	0.00	0.00	0.0	0.0	0.47
D2	0.49	0.0	0.38	0.29	0.38	0.00	0.38	0.49	0.0	0.0	0.00
D3	0.00	0.5	0.00	0.30	0.00	0.00	0.38	0.00	0.5	0.5	0.00

Efecto del IDF dentro de D1: "el" aparece 2 veces y obtiene 0.55; "gato" aparece solo 1 vez pero obtiene 0.47 — repetirse el doble apenas compensa ser una palabra común.

Bag-of-Words frente a TF-IDF

Misma fórmula de coseno, dos representaciones distintas del mismo corpus.

```
from sklearn.feature_extraction.text import CountVectorizer

def similitud_coseno(a, b):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return a.dot(b) / (np.linalg.norm(a) * np.linalg.norm(b))

count_vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X_counts = count_vectorizer.fit_transform(corpus.values()).toarray()
X_tfidf_arr = X_tfidf.toarray()
```

```
cos_bow = {
    "D1-D2": similitud_coseno(X_counts[0], X_counts[1]),
    "D1-D3": similitud_coseno(X_counts[0], X_counts[2]),
}
cos_tfidf = {
    "D1-D2": similitud_coseno(X_tfidf_arr[0], X_tfidf_arr[1]),
    "D1-D3": similitud_coseno(X_tfidf_arr[0], X_tfidf_arr[2]),
}

pd.DataFrame({"Bow": cos_bow, "TF-IDF": cos_tfidf}).round(3)
```

	Bow	TF-IDF
D1-D2	0.577	0.430
D1-D3	0.316	0.165

```
sep_bow = cos_bow["D1-D2"] / cos_bow["D1-D3"]
sep_tfidf = cos_tfidf["D1-D2"] / cos_tfidf["D1-D3"]
print(f"Separación con BoW: {cos_bow['D1-D2']:.3f} / {cos_bow['D1-D3']:.3f} = {sep_bow:.1f}x")
print(f"Separación con TF-IDF: {cos_tfidf['D1-D2']:.3f} / {cos_tfidf['D1-D3']:.3f} = {sep_tfidf:.1f}x")
```

Separación con BoW: 0.577 / 0.316 = 1.8x
 Separación con TF-IDF: 0.430 / 0.165 = 2.6x

Con conteos, D1 y D3 (temas sin relación) todavía comparten 0.316 de similitud solo por culpa de "el". Con TF-IDF esa similitud cae a 0.165, mientras que el par relacionado (D1-D2) se mantiene alto: la separación entre "parecido" y "no parecido" casi se duplica. Esa distancia es la que sirve en un buscador real.

Ejercicio 01 — Vectorización dispersa

Objetivo: ¿cuánta información se pierde al ignorar el orden de las palabras, y cuánto ayuda TF-IDF a rankear por relevancia? Se trabaja sobre los 619 comentarios reales de clientes.

```
import spacy

nlp = spacy.load("es_core_news_sm")

def normalizar_muchos(textos):
    return [
        " ".join(t.lemma_ for t in doc if t.is_alpha and not t.is_stop)
        for doc in nlp.pipe((str(x).lower() for x in textos), batch_size=64)
    ]

comentarios = pd.read_csv("../data/comentarios.csv")
comentarios["texto_normalizado"] = normalizar_muchos(comentarios["texto_comentario"])
print(f"{len(comentarios)} comentarios normalizados")
```

620 comentarios normalizados

1-2. Matriz término-documento y dispersidad real

```

cv = CountVectorizer(min_df=2)
X_cv = cv.fit_transform(comentarios["texto_normalizado"])

n_docs, n_vocab = X_cv.shape
dispersidad = 1 - X_cv.nnz / (n_docs * n_vocab)
print(f"CountVectorizer: {n_docs} documentos x {n_vocab} términos")
print(f"Dispersidad: {dispersidad:.1%} de la matriz son ceros")

```

CountVectorizer: 620 documentos x 514 términos
 Dispersidad: 97.9% de la matriz son ceros

3-4. Documentos más parecidos: BoW vs TF-IDF

Se elige un comentario cualquiera y se buscan los 3 más parecidos con cada representación.

```

from sklearn.metrics.pairwise import cosine_similarity

tfidf = TfidfVectorizer(min_df=2)
X_tf = tfidf.fit_transform(comentarios["texto_normalizado"])

idx_consulta = 0
print("Consulta:", comentarios.loc[idx_consulta, "texto_comentario"][:100], "...\\n")

for nombre, matriz in [("BoW", X_cv), ("TF-IDF", X_tf)]:
    sims = cosine_similarity(matriz[idx_consulta], matriz).ravel()
    top = sims.argsort()[::-1][1:4] # excluye el propio documento
    print(f"-- Top 3 más parecidos según {nombre} --")
    for i in top:
        print(f"    [{sims[i]:.3f}] {comentarios.loc[i, 'texto_comentario'][:90]}...")
    print()

```

Consulta: El Smartphone Nexus 5G es un cambio de juego. La pantalla OLED es vívida, la cámara de 108MP captura ...

-- Top 3 más parecidos según BoW --

[0.503] El Smartphone Nexus 5G es rápido y la pantalla es brillante, pero la batería se drena un p...

[0.470] El Smartphone Nexus es muy rápido y la pantalla es de una calidad impresionante. La cámara...

[0.450] El Smartphone Nexus es bueno en general. Sin embargo, la batería se agota rápidamente con ...

-- Top 3 más parecidos según TF-IDF --

[0.415] El Smartphone Nexus es muy rápido y la pantalla es de una calidad impresionante. La cámara...

[0.361] El Smartphone Nexus 5G es rápido y la pantalla es brillante, pero la batería se drena un p...

[0.348] El Smartphone Nexus es bueno en general. Sin embargo, la batería se agota rápidamente con ...

Reto — ngram_range=(1, 2)

Contar también pares de palabras (bigramas) recupera algo de orden, al costo de un vocabulario mucho más grande.

```
import time

for ngram_range in [(1, 1), (1, 2)]:
    inicio = time.perf_counter()
    vect = CountVectorizer(min_df=2, ngram_range=ngram_range)
    X_ng = vect.fit_transform(comentarios["texto_normalizado"])
    duracion = time.perf_counter() - inicio
    print(f"ngram_range={ngram_range}: |V| = {X_ng.shape[1]:>5}
    ({duracion*1000:.0f} ms)")
```

```
ngram_range=(1, 1): |V| = 514 (4 ms)
ngram_range=(1, 2): |V| = 1532 (6 ms)
```

El vocabulario crece varias veces al incluir bigramas, y la mayoría de esos pares nuevos aparece una sola vez en todo el corpus (son demasiado específicos para generalizar). En un corpus de 619 comentarios el costo en tiempo todavía es insignificante; con miles de documentos deja de serlo. El ranking de similitud tiende a moverse poco: la mayoría de la señal ya la aportan las palabras sueltas.

Siguiente: `04_word2vec.ipynb` — vectores densos que aprenden del contexto en lugar de contar.